```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Load the Titanic dataset from the public CSV source.
df = pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv')

# Keep rows with the values needed for fare prediction so the target has no missing values.
df.dropna(subset=['Age', 'Fare', 'Pclass'], inplace=True)

# Select input features and the continuous value that the regression model will predict.
features = ['Age', 'Pclass']
target_reg = 'Fare'
X_reg = df[features]
y_reg = df[target_reg]

# Split the rows into training and testing sets; the seed makes the split reproducible.
X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(
    X_reg, y_reg, test_size=0.2, random_state=42
)
```

```
# Create the regression model and fit it to the training data.
lr_model = LinearRegression()
lr_model.fit(X_train_reg, y_train_reg)

# Predict fares for the test set, which the model did not train on.
y_pred_reg = lr_model.predict(X_test_reg)

# Calculate MSE and RMSE; RMSE is in fare units and is easier to interpret.
mse = mean_squared_error(y_test_reg, y_pred_reg)
print(f'Mean Squared Error for Fare Prediction: {mse:.2f}')
print(f'Root Mean Squared Error: {np.sqrt(mse):.2f}')
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Convert the Sex text labels into numeric values that the model can use.
df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})

# Keep rows with all required classification values, then select features and target.
df_cls = df.dropna(subset=['Age', 'Pclass', 'Sex', 'Survived']).copy()
features_cls = ['Age', 'Pclass', 'Sex']
target_cls = 'Survived'
X_cls = df_cls[features_cls]
y_cls = df_cls[target_cls]

# Use a reproducible split so evaluation uses passengers held out from training.
X_train_cls, X_test_cls, y_train_cls, y_test_cls = train_test_split(
    X_cls, y_cls, test_size=0.2, random_state=42, stratify=y_cls
)
```

```
# Create the classifier and allow extra iterations for reliable convergence.
log_model = LogisticRegression(max_iter=1000)
log_model.fit(X_train_cls, y_train_cls)

# Predict survival labels for the held-out test data.
y_pred_cls = log_model.predict(X_test_cls)

# Accuracy is the share of test predictions that match the true labels.
accuracy = accuracy_score(y_test_cls, y_pred_cls)
print(f'Accuracy for Survival Prediction: {accuracy:.2%}')
```